Group 2 — Systems Integration Project

HMS ↔ LIS Integration

Hospital Management System ⇄ Laboratory Information System — API-Based Integration

Django 6 · Django REST Framework · JWT Authentication · PostgreSQL 16 (Docker) · Web Dashboard

1. What This Project Does

This project demonstrates a complete, real-world integration between a **Hospital Management System (HMS)** and a **Laboratory Information System (LIS)**. The HMS submits a laboratory test request over HTTP; the LIS validates, accepts, and processes it; the HMS then retrieves the patient's laboratory result. Both systems run as separate Django apps that communicate **only through a public REST API authenticated with JWT** — exactly as two independent hospital systems would.

Requirement	How it is fulfilled
Request validation	Missing fields, unknown patient, bad UUID, future date-of-birth, blank values → HTTP 400. Unknown/inactive test code → HTTP 422.
Authentication	JWT via `/api/auth/token/` (SimpleJWT). Every API call requires `Authorization: Bearer <token>`. No token → HTTP 401.
Authorization	Separate service accounts: `hms_demo` (submits requests / retrieves results) and `lab_admin` (lab staff — records results, admin panel).
Response handling	The HMS-side client (`integration/lis_client.py`) maps every LIS response (200/201/400/401/404/409/422/5xx) into HMS request status transitions: PENDING → SENT REJECTED ERROR COMPLETED
Error handling	Connection failures, auth failures (with one automatic token-refresh retry), validation rejections, DB failures wrapped in `transaction.atomic()`, idempotent duplicate handling — always with a clean JSON error body, never a stack trace.
Transaction logging	Every inbound and outbound exchange is persisted in the `integration_integrationlog` PostgreSQL table — including errors — visible in the dashboard, admin, API, or CLI.

2. Architecture

```
[ db ]  PostgreSQL 16  (volume: pgdata)
[ web ] Django + gunicorn + dashboard  (port 8000)
```

3. Requirements to Run

Requirement	Check	Notes
Docker + Docker Compose	`docker compose version`	The only hard requirement — everything else runs in containers.
Free ports	—	Port **8000** (web app) and **5432** (Postgres) must be free. Stop conflicting containers (e.g. other DBs) if needed.
(Optional) Python 3.12+ on host	`python --version`	Only needed to run the CLI demo script or tests from the host machine.

Before you start: if another application (for example a Nextcloud stack or a local PostgreSQL service) already occupies port 5432 or 8000, stop it first, or change the port mapping in `docker-compose.yml` (e.g. `"5433:5432"` and `"8001:8000"`) — if you change 8000, also update `LIS_API_BASE_URL` and the dashboard URLs accordingly.

4. Running the Project (Docker — Recommended)

```
# 1. From the project root (the folder containing docker-compose.yml):
docker compose up --build

#    The web container automatically:
#      - waits for PostgreSQL to become healthy
#      - applies all migrations
#      - seeds the lab test catalog, service accounts and demo patients
#      - starts gunicorn on 0.0.0.0:8000

# 2. Open the dashboard in your browser:
http://localhost:8000/

# 3. To stop:
docker compose down          # keeps data
docker compose down -v       # also deletes the database volume
```

Useful commands

```
docker compose ps                                  # container status
docker compose logs -f web                         # live application logs
docker compose restart web                         # restart the app
docker compose exec web python manage.py transaction_log -n 30   # view audit trail
docker compose exec web python manage.py test integration        # run the test suite
docker compose exec web python manage.py createsuperuser         # extra admin user
```

5. Running the Demonstration

A Browser dashboard (recommended for presentations)

1. Open **http://localhost:8000/**
2. Click ▶ **Run Full Demo (auto)** — it walks through all 8 steps with a small pause between each, while the three tables below update live.
3. What you will see, in order:
   - **Validation errors** — missing fields and unknown patient → HTTP 400
   - **Unauthenticated call** rejected → HTTP 401
   - **Valid submission** — HMS creates the request, LIS accepts it and returns an order number (e.g. `LIS-20260908-F1091823`) → HTTP 201
   - **Result before analysis** → HTTP 409 “Result not available yet”
   - **Lab staff records the result** — order becomes COMPLETED → HTTP 201
   - **HMS retrieves the result** — full result payload → HTTP 200
   - **Unknown test code** — LIS rejects, HMS marks request REJECTED → HTTP 422
   - **Idempotent re-submission** — the same `request_id` returns the same order, no duplicate → HTTP 200

4. The **Integration Transaction Log** table at the bottom shows every request/response exchange (including the failures — that is intentional: it proves logging works).
5. You can also drive each step manually: pick a patient + test → **Submit**, then select the request → **Record Result**, and use the **result** buttons in the LIS table.

B CLI script

```
# From the host (server must be running):
python simulate_integration.py

# Or from inside the container:
docker compose exec -e BASE_URL=http://web:8000 web python simulate_integration.py
```

6. API Reference

Method	Endpoint	Auth	Purpose
GET	`/`	—	Dashboard frontend (demo UI)
POST	`/api/auth/token/`	—	Obtain JWT (`username` , `password`) → `access` , `refresh`
POST	`/api/auth/token/refresh/`	refresh token	Refresh JWT
GET / POST	`/api/hms/patients/`	JWT	List / create patients (HMS)
GET / POST	`/api/hms/lab-requests/`	JWT	List / submit lab test requests (HMS → LIS). Optional `request_id` for idempotency.
GET	`/api/hms/lab-requests/<request_id>/result/`	JWT	HMS retrieves the result via the LIS client (409 until ready)
GET / POST	`/api/lis/orders/`	JWT	List orders / ingest an order (idempotent on `request_id`)
GET	`/api/lis/orders/<request_id>/`	JWT	Order status (includes result once complete)
POST	`/api/lis/orders/<request_id>/process/`	JWT	Lab staff record the result; order → COMPLETED
GET	`/api/lis/orders/<request_id>/result/`	JWT	LIS-side result retrieval (409 until ready, 404 unknown)
GET	`/api/lis/catalog/`	JWT	Active lab test catalog
GET	`/api/logs/?limit=50`	JWT	Integration transaction log (JSON)
GET	`/admin/`	staff login	Django admin (browse all data + logs)

Example: end-to-end with curl

```
# 1. Get a token (HMS service account)
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/token/ \
  -H "Content-Type: application/json" \
  -d '{"username":"hms_demo","password":"HmsDemo#2024"}' | jq -r .access)

# 2. Submit a lab request (HMS → LIS)
curl -s -X POST http://localhost:8000/api/hms/lab-requests/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
```

```
  -d '{"patient_number":"HMS-0001","test_code":"CBC","test_name":"Complete Blood Count"}'
# → 201 {"detail":"...","request_id":"...","status":"SENT","lis_order_number":"LIS-..."}

# 3. Retrieve the result (409 until lab staff process it, 200 afterwards)
curl -s http://localhost:8000/api/hms/lab-requests/<request_id>/result/ \
  -H "Authorization: Bearer $TOKEN"
```

7. Test Accounts & Seed Data

Username	Password	Role
`lab_admin`	`LabAdmin#2024`	Lab staff — records results, Django admin
`hms_demo`	`HmsDemo#2024`	HMS service account (used by the integration client)

Seed patients: `HMS-0001` Amina Juma (F, 1990) · `HMS-0002` Baraka Mushi (M, 1985) · `HMS-0003` Neema Kileo (F, 2001)
Seed catalog: CBC, LFT, RFT, BS, LIP, URM

8. Running the Tests

```
docker compose exec web python manage.py test integration
```

Expected result: *Ran 14 tests … OK* — covers authentication (401 / token issuance), validation (missing fields, unknown patient, bad UUID), order ingestion, unknown test code (422), future DOB (400), idempotency, result lifecycle (409 → 200), unknown request (404), and transaction logging.

9. Troubleshooting & Fixed Issues

Issue	Cause	Solution / What was fixed
`web` container fails: `entrypoint.sh: no such file`	Entrypoint missing from the image	Fixed in `Dockerfile` (`COPY entrypoint.sh` + `chmod +x`). Rebuild: `docker compose up --build` .
“LIS authentication failed” / HTTP 400 on submission inside Docker	Container-to-container calls use hostname `web` , which was not in `ALLOWED_HOSTS` (also, the project `.env` overrides compose defaults)	`ALLOWED_HOSTS` now includes `web` in **both** `.env` / `.env.example` and `docker-compose.yml` . After changing env vars run: `docker compose up -d --force-recreate web` .
Duplicate submission raised HTTP 500 (IntegrityError)	Re-submitting the same `request_id` violated the unique constraint	HMS view now treats a repeated `request_id` idempotently (returns the existing order, HTTP 200); the client never downgrades a COMPLETED request.

Port already allocated (5432 / 8000)	Another stack (e.g. Nextcloud/MariaDB, local Postgres) uses the port	Stop the other container, or change the port mapping in `docker-compose.yml`.
`Database "healthms" does not exist` on first run	Postgres volume not initialised yet	Not an issue with compose — the `db` service creates it automatically from `POSTGRES_DB`. Just wait for the healthcheck.
Want a clean re-demo	Data persists in the `pgdata` volume	`docker compose exec web python manage.py flush --noinput && docker compose restart web` (entrypoint re-seeds), or `docker compose down -v` for a full reset.
Dashboard shows “Cannot reach API”	Web container still starting / crashed	`docker compose logs -f web` — wait for *“Booting worker”*, then reload the page.

10. Project Structure

```
├── docker-compose.yml         # db + web services
├── Dockerfile / entrypoint.sh # app image, auto-migrate + seed
├── requirements.txt           # Django, DRF, SimpleJWT, psycopg2, gunicorn, whitenoise
├── .env / .env.example        # all configuration (DB, LIS URL, service credentials)
├── manage.py
├── healthMS/                  # project settings & root URL config
├── hms/                       # HOSPITAL system: Patient, LabRequest, submission & result
views
│   └── management/commands/seed_demo_data.py
├── lis/                       # LAB system: catalog, order ingest (idempotent), process,
results
├── integration/               # LISClient (JWT, retries), IntegrationLog, log API, tests
├── templates/dashboard.html   # the web dashboard
├── simulate_integration.py    # CLI end-to-end demo
└── docs/project_guide.html    # this guide (source of the PDF)
```